

Multimedia Information Processing, Spring 2026

Final Project Report

MoodWave: Dynamic Music Emotion Recognition with Audio-Reactive Visualization

Group Number: 4

Team Members: Vincent Ma, Dhana Kresnawijaya

Supervisor: Jimmy Liu

Submission Date: 7 June 2026

Abstract

MoodWave is a multimedia system that listens to a music track, estimates how its emotion changes over time, and turns those estimates into an audio-reactive visual experience. Instead of assigning one mood label to an entire song, the system predicts valence and arousal every 0.5 seconds. This makes the output suitable for visual effects that need to change when the music changes. The final backend supports several encoder and classifier combinations, including Mel/MFCC with CNN-BiGRU, OpenL3 with MLP, CLAP with MLP, and MuQ with both MLP and BiGRU classifiers. The strongest implemented model is MuQ-BiGRU. It represents each song as an ordered sequence of 0.5-second MuQ embeddings and predicts a valence-arousal pair for each window. On the strict track-level DEAM split, MuQ-BiGRU achieves valence $R^2 = 0.603$, arousal $R^2 = 0.680$, and quadrant accuracy 67.4%. The deployed project connects this backend to a browser frontend and a TouchDesigner visualization. The frontend handles upload, progress display, saved-analysis management, playback, and the live TouchDesigner preview, while TouchDesigner receives OSC timeline and stem signals to animate a flower-and-particle scene. Demucs six-source separation provides vocals, drums, bass, guitar, piano, and other stems so different visual components can respond to different musical sources.

Contents

1	Introduction	3
2	Dataset and Problem Definition	3
2.1	DEAM Dynamic Annotations	3
2.2	Prediction Task	4
3	System Architecture	4
3.1	Overall Pipeline	4
3.2	Unified Training Interface	5
4	Methodology	6
4.1	Baseline Models	6
4.2	MuQ Window Embedding Extraction	7
4.3	MuQ-MLP	7
4.4	MuQ-BiGRU	8
4.5	Optimization	8
4.6	Demucs Stem Separation	9
5	Implementation Details	10
5.1	Backend Organization	10
5.2	Memory Management	11
5.3	API and Analysis Streaming	11
5.4	Frontend Interface	11
5.5	Saved Analysis Library	12
5.6	OSC Bridge and TouchDesigner Input	12
5.7	TouchDesigner Visual Mapping	13
5.8	TouchDesigner Render Streaming	14

6	Experiments and Results	14
6.1	Experimental Setup	14
6.2	Quantitative Results	15
6.3	Discussion	17
7	Limitations and Future Work	18
7.1	Deployment-Time MuQ Memory	18
7.2	Train-Inference Consistency	18
7.3	Hyperparameter Search Reliability	18
7.4	Cross-Dataset Evaluation	19
7.5	Frontend and TouchDesigner Synchronization	19
7.6	Visualization Evaluation	19
8	Conclusion	19

1 Introduction

Music emotion recognition (MER) is difficult because music does not express emotion through one simple signal. A listener’s perception of mood may depend on loudness, rhythm, harmony, timbre, the shape of a melody, and how the song develops over time. For this reason, a practical MER system should not treat a whole song as if it has only one emotional state. A song can start calmly, become more energetic in the chorus, and later return to a darker or more reflective mood. MoodWave is designed around this idea: it estimates an emotion timeline rather than a single label.

The project uses Russell’s valence-arousal model [1]. Valence describes how positive or negative the emotion feels, while arousal describes how intense or calm it feels. For example, happy music usually has positive valence and medium-to-high arousal, while sad music usually has negative valence and lower arousal. In MoodWave, every 0.5-second audio window receives a predicted pair (v_t, a_t) . These values can then be mapped to visual behavior, such as brighter colors for positive valence or faster particle motion for high arousal.

The final implementation focuses on three goals:

- O1.** Build a reliable MER backend that evaluates models on unseen songs, not on leaked segments from songs already seen during training.
- O2.** Produce window-level predictions so the visualization can react to emotional changes inside a track.
- O3.** Organize the code so different audio encoders and classifiers can be compared without maintaining many separate training scripts.
- O4.** Integrate the model output with a usable frontend and TouchDesigner pipeline so the analysis can be viewed, replayed, saved, and used for real-time visual control.

2 Dataset and Problem Definition

2.1 DEAM Dynamic Annotations

The main dataset used in this project is DEAM, which contains 1,802 songs with continuous valence and arousal annotations [2]. DEAM is well suited to MoodWave because it provides dynamic labels instead of only one label per song. The annotations are sampled every 0.5 seconds, so each valid timestamp becomes one supervised training target:

$$y_t = (v_t, a_t), \quad v_t, a_t \in [-1, 1], \quad (1)$$

where t refers to a 0.5-second window. The preprocessing code aligns these timestamps with the audio and removes windows that do not have valid labels.

A key part of the evaluation is the split strategy. The project uses a strict song-level split, meaning all windows from one song stay together in either training, validation, or testing. This avoids a common mistake in audio tasks: if nearby windows from the same song are placed in both training and testing, the model may recognize the song rather than learn general emotion patterns. Strict splitting gives lower but more honest metrics.

2.2 Prediction Task

The learning problem is treated as two-output regression. For each 0.5-second window, the model predicts:

$$\hat{y}_t = (\hat{v}_t, \hat{a}_t). \quad (2)$$

Regression quality is measured using mean absolute error (MAE) and coefficient of determination (R^2):

$$R^2 = 1 - \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}. \quad (3)$$

For easier interpretation, the continuous predictions are also converted into emotion quadrants. This gives classification-style metrics such as quadrant accuracy, weighted F1, and macro F1. These metrics do not replace MAE and R^2 , but they help explain whether the model places a window in the correct emotional region.

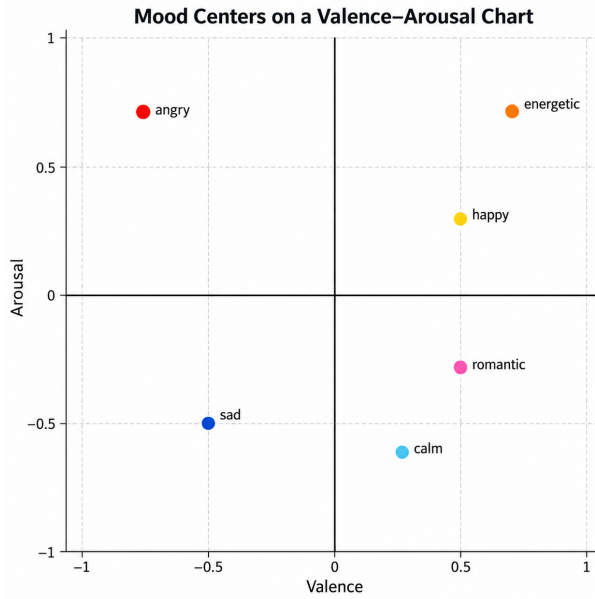


Figure 1: Conceptual valence-arousal emotion space used by MoodWave.

3 System Architecture

3.1 Overall Pipeline

MoodWave is organized as three connected layers: a Python analysis backend, a browser-based control frontend, and a TouchDesigner visual runtime. The backend is responsible for audio processing and machine learning inference. The frontend is responsible for user interaction, progress display, saved-analysis management, browser audio playback, and forwarding synchronized data to TouchDesigner. TouchDesigner is responsible for rendering the final flower-and-particle visualization.

The backend is divided into four main parts:

- **Feature extraction:** loads audio and converts it into model inputs such as Mel/M-FCC features, OpenL3 embeddings, CLAP embeddings, or MuQ embeddings.

- **Emotion prediction models:** contains the neural networks that predict valence and arousal, including simple MLP baselines and the final MuQ-BiGRU sequence model.
- **Training and evaluation:** handles dataset splitting, model training, metric logging, checkpoint saving, plot generation, and optional Lion Swarm Optimization (LSO).
- **Serving:** provides FastAPI endpoints for uploading audio, separating stems, streaming mood predictions, saving analyses, and serving the frontend.

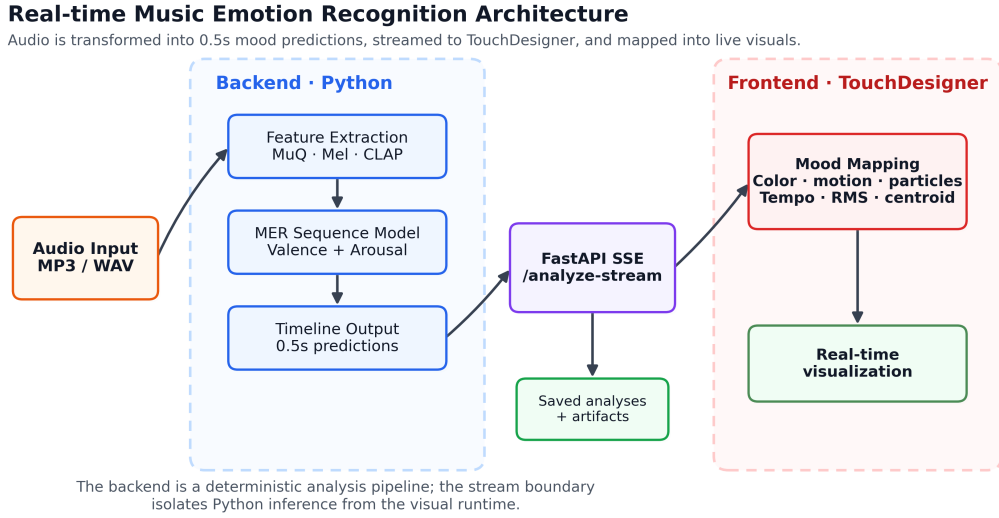


Figure 2: Final MoodWave system architecture.

The FastAPI service exposes an `/analyze-stream` endpoint. A user uploads an audio file, and the backend converts it to a stable WAV format so the frontend and TouchDesigner can read it reliably. The backend separates the audio into six stems using Demucs and then runs the MuQ-BiGRU inference pipeline to produce a fine 0.5-second emotion timeline. During this process, progress messages and prediction results are streamed back with server-sent events (SSE). SSE is a good fit here because the communication is mostly one-way: the backend sends analysis updates to the client.

After the analysis finishes, the frontend will send one complete timeline payload to the `touch_designer/osc_bridge.py` over WebSocket. The bridge converts that payload into OSC messages for TouchDesigner. This keeps model inference outside TouchDesigner while still giving TouchDesigner the data it needs for real-time control. TouchDesigner then renders the visualization and exposes its final TOP through a local MJPEG server at `http://localhost:9000/video`, which the frontend displays inside the main visual panel.

3.2 Unified Training Interface

Earlier versions of the backend used separate training files for different model combinations. For example, one script trained MuQ-MLP while another trained OpenL3-MLP.

Table 1: Runtime communication channels used by the final MoodWave application.

Channel	Direction	Purpose
HTTP/SSE	Backend → frontend	Streams analysis progress, chunk summaries, final timeline, file path, and stem paths through <code>/analyze-stream</code> .
WebSocket	Frontend → OSC bridge	Sends one batched timeline payload after analysis and sends play, pause, and seek commands during playback.
OSC UDP	OSC bridge → TouchDesigner	Loads audio files and stem files, writes timeline arrays into OSC In CHOP/DAT operators, and controls transport.
MJPEG HTTP	TouchDesigner → frontend	Streams the rendered TouchDesigner output back to the browser preview panel.

This became hard to maintain because shared logic, such as splitting, evaluation, plotting, and checkpoint saving, had to be repeated. The final version uses one training entry point: `backend/train.py`. The experiment is selected with command-line options:

```
python train.py --encoder muq --classifier bigru --split strict
python train.py --encoder muq --classifier mlp --split strict
python train.py --encoder mel --classifier cnn-bigru --split strict
python train.py --encoder openl3 --classifier mlp --split paper --eval-only
```

This design makes the code easier to extend. The encoder decides how audio is represented, while the classifier decides how that representation is used for prediction. The script also saves outputs consistently: checkpoints go to `models/weights/`, and reports, metric CSVs, and plots go to `models/metrics/`. When normalization statistics are needed for inference, they are embedded in the checkpoint instead of being saved as a separate loose file.

4 Methodology

4.1 Baseline Models

The first backend models were useful baselines. The CNN-BiGRU model used Mel/MFCC spectrogram windows as input. Its convolution layers learned local frequency patterns, while the BiGRU tried to model temporal structure inside each segment. The OpenL3-MLP baseline used pre-trained audio embeddings [3] and a small feed-forward network for prediction.

These baselines helped shape the final approach. First, they showed that strict song-level splitting is necessary for a fair evaluation. Segment-level splitting produced much better numbers, but those numbers were not reliable because the same songs could appear in both training and testing. Second, they showed that pre-trained audio representations were stronger than features learned from scratch on DEAM alone. This motivated the final focus on MuQ, a large self-supervised music representation model designed for music audio [5].

4.2 MuQ Window Embedding Extraction

MuQ expects 24 kHz mono audio and produces hidden representations from a music-focused encoder. In this project, MuQ is used as the feature extractor. The training pipeline creates one embedding for each 0.5-second annotation window. Conceptually:

$$x_t = \text{Pool}(\text{MuQ}(\text{audio})_{t:t+0.5s}), \quad (4)$$

where x_t is the embedding aligned with target (v_t, a_t) . Running MuQ is computationally expensive, so the extracted embeddings are cached. This allows the classifier experiments to run quickly without repeatedly recomputing the same audio features.

The same MuQ embeddings can be used in two ways:

- **Vector form:** each 0.5-second embedding is treated as an independent sample. This is used by MuQ-MLP.
- **Sequence form:** embeddings from the same song are sorted by timestamp and grouped into one sequence. This is used by MuQ-BiGRU.

MuQ window extraction and DEAM label alignment

Each 0.5s audio window becomes one embedding, which keeps the timestamp of its valence-arousal label.

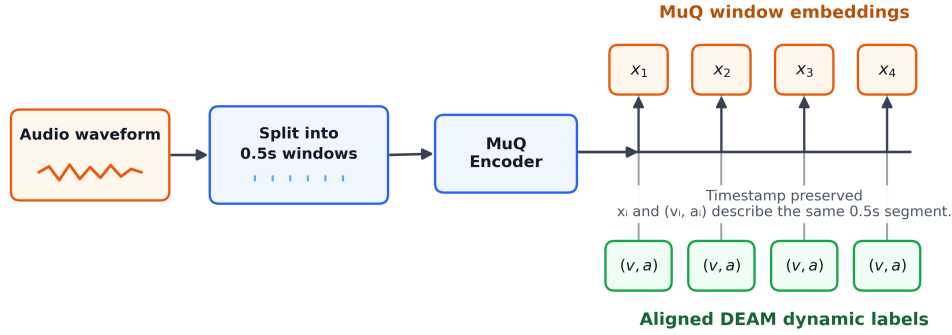


Figure 3: MuQ window-level feature extraction and label alignment.

4.3 MuQ-MLP

The MuQ-MLP baseline treats each window independently:

$$(\hat{v}_t, \hat{a}_t) = f_\theta(x_t). \quad (5)$$

This model is simple and fast. It is also a strong baseline because MuQ already provides rich musical features. However, MuQ-MLP sees each window separately. It does not know what happened before or after the current window. This is a limitation for music, where emotional meaning often depends on context. For example, a short rising section may feel different depending on whether it is part of a chorus build-up or a transition into a calmer part of the song.

4.4 MuQ-BiGRU

The final model fixes this limitation by keeping the windows in time order. This is where the BiGRU is useful. A GRU, or gated recurrent unit, is a neural network layer designed for sequential data. It reads one time step after another and keeps a hidden state that summarizes useful information from nearby context. A *bidirectional* GRU reads the sequence in both directions: forward from the beginning of the song and backward from the end. This allows each window prediction to use information from both earlier and later parts of the song.

Each song becomes a sequence:

$$X_i = [x_1, x_2, \dots, x_{T_i}], \quad (6)$$

where T_i is the number of valid 0.5-second windows in song i . A bidirectional GRU reads the sequence and produces one prediction for every window:

$$H_i = \text{BiGRU}(X_i), \quad (\hat{v}_t, \hat{a}_t) = g_\theta(H_{i,t}). \quad (7)$$

Songs have different numbers of valid windows, so shorter sequences are padded to fit into batches. Padding masks ensure that these artificial positions do not affect the loss or metrics. This architecture matches the goal of MoodWave: the output is a mood timeline, not one average mood for the entire track.

BiGRU sequence model over ordered MuQ embeddings

Bidirectional context lets each prediction use information from earlier and later windows in the song.

Ordered 0.5s MuQ embeddings



The model reads the entire sequence before predicting, so each 0.5s output is conditioned on the surrounding musical context.

Figure 4: MuQ-BiGRU dynamic sequence architecture.

4.5 Optimization

The models are trained with AdamW, early stopping, and validation monitoring. The training script also includes Lion Swarm Optimization (LSO), which searches over hyperparameters such as learning rate, weight decay, batch size, hidden size, layer count, and dropout.

To understand how LSO works, LSO treats each possible hyperparameter setting as one candidate solution. A candidate may contain values such as learning rate, batch

size, dropout, hidden size, and number of BiGRU layers. The algorithm starts with a population of candidates, trains each candidate for a short evaluation run, and scores it on the validation set.

After scoring, the better candidates guide the next search step. Weaker candidates are moved toward more promising regions of the search space, while the algorithm still keeps some variation so it can explore new combinations. This process repeats for several iterations. At the end, the best candidate found by LSO is used as the selected hyperparameter configuration for full training.

LSO is useful as an exploration tool, but it is not automatically better than a carefully chosen baseline. In this project, very large LSO searches sometimes selected parameters that looked good during validation search but performed worse on the final test set. This can happen when many candidates are compared on one validation split or when candidate models are trained for fewer epochs than the final model. For that reason, this report focuses on the best reliable strict-split test results instead of assuming that the largest search run must be the best model.

4.6 Demucs Stem Separation

In addition to predicting valence and arousal, MoodWave separates the uploaded song into musical stems using Demucs [6]. The deployed backend uses the `htdemucs_6s` model, which estimates six sources from the same mixture: vocals, drums, bass, guitar, piano, and other. This is important for the visualization because different stems describe different kinds of musical events. The mood model gives the emotional state of the song, while the stems give instrument-level control signals.

The stem separation pipeline first converts the uploaded audio into a format that Demucs can process, runs split inference with overlap to reduce memory pressure, and then writes each estimated source as a TouchDesigner-friendly 44.1 kHz stereo PCM WAV file. The final stem paths are sent to the frontend and then forwarded to TouchDesigner through OSC. TouchDesigner can load each stem into a separate Audio File In CHOP and analyze it independently.

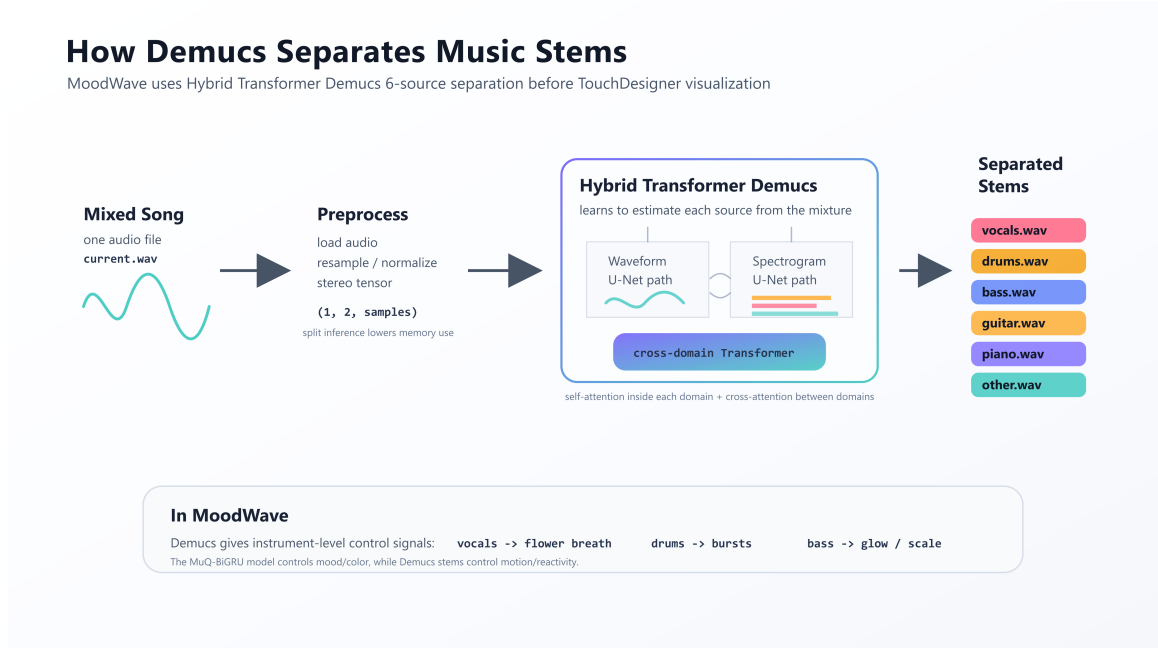


Figure 5: Demucs six-stem separation pipeline used for instrument-level visual control.

The separation is used as a control layer, not as a replacement for the MER model. For example, the vocals stem can drive the flower center glow, drums can drive particle bursts, bass can drive scale and bloom, guitar can drive swirl or turbulence, piano can drive shimmer, and the other stem can drive slower background atmosphere. This creates a richer visual system than using only one mixed-audio amplitude signal.

5 Implementation Details

5.1 Backend Organization

The final repository structure is organized around responsibilities:

- `backend/core/model.py`: defines the neural network architectures.
- `backend/features/`: extracts Mel/MFCC features and pre-trained embeddings from MuQ, CLAP, and OpenL3.
- `backend/training/`: contains reusable training loops, evaluation code, plotting utilities, and LSO search.
- `backend/train.py`: provides one command-line interface for all training experiments.
- `backend/main.py`: runs the FastAPI service for upload, analysis, streaming, and saved results.

This structure keeps the project maintainable. New encoders or classifiers can be added without copying an entire training file. It also helps ensure that different models are evaluated using the same metrics and split logic.

5.2 Memory Management

MuQ is much heavier than the MLP or BiGRU classifier that comes after it. This distinction is important. Even if the final classifier is lightweight, the feature extractor can still use a large amount of GPU memory. During cache generation, the extractor clears CUDA memory after each song and can retry failed songs on CPU after a CUDA out-of-memory error.

For full-length user songs, the safer deployment strategy is chunked extraction. Instead of sending a 2–3 minute waveform through MuQ in one forward pass, the backend should process shorter overlapping chunks, pool 0.5-second embeddings, concatenate the embeddings on CPU, and then run the MuQ-BiGRU model over the final sequence. This keeps memory bounded while still giving the BiGRU a full timeline to analyze.

5.3 API and Analysis Streaming

The inference API streams progress and results through SSE. Each response can include average valence, average arousal, a mood distribution, basic audio features such as tempo, energy, and brightness, and a timeline of local dynamic predictions. The backend first sends the converted audio file path and total duration, then sends separation and analysis status messages, then sends 10-second chunk summaries for the browser interface, and finally sends the full MuQ-BiGRU 0.5-second timeline. This gives the user immediate progress feedback even though stem separation and model inference take time.

The backend writes the current upload into a run-specific upload directory and converts it to `current.wav`. This is the file path sent to TouchDesigner. Stem separation writes `vocals.wav`, `drums.wav`, `bass.wav`, `guitar.wav`, `piano.wav`, and `other.wav` under the same run directory. The frontend receives these absolute paths, stores them with the current analysis, and sends them to the OSC bridge.

5.4 Frontend Interface

The final frontend is implemented in `frontend/moodwave.html`. It is a single-page browser interface with four main responsibilities:

- **Upload and progress display:** the user can drag and drop or select an audio file. The page posts the file to `/analyze-stream` and reads the SSE stream with a browser `ReadableStream`. A separation overlay shows progress during Demucs processing, and the main progress bar updates while chunk summaries arrive.
- **Mood dashboard:** the sidebar displays the current dominant mood, six mood-probability bars, tempo, energy, brightness, a valence-arousal plot, and a scrollable chunk timeline.
- **Playback control:** the browser audio element is the timing reference. The play button, progress bar, seeking, and spacebar shortcut update the browser audio time and send synchronized play, pause, or seek messages to TouchDesigner.
- **TouchDesigner preview:** the main visual panel displays the MJPEG stream produced by TouchDesigner and shows an offline state when the TouchDesigner server is not yet available.

During playback, the frontend interpolates between nearby timeline points for smooth UI display. The mood bars, feature cards, V/A marker, and active chunk list update every animation frame. The TouchDesigner timeline itself is sent as one batch after analysis so TouchDesigner does not depend on the browser sending a new mood value every frame.

5.5 Saved Analysis Library

The frontend includes a save modal and a library modal so users do not need to repeat slow stem separation every time they want to use the same song. When the user saves an analysis, the frontend sends the analysis name, source filename, audio path, chunk list, stem paths, and a compact V/A path snapshot to `/saved-analyses`. The backend copies the current audio file and any available stems into `backend/saved_analyses/` and writes a `metadata.json` file.

The library view lists saved songs, shows the selected song's summary statistics, displays average mood distribution bars, shows saved stem tags, and draws the saved V/A path on a canvas. A saved analysis can be loaded back into the frontend and immediately sent to TouchDesigner without re-running Demucs or MuQ-BiGRU. The library also supports deletion through the backend `DELETE /saved-analyses/{id}` endpoint.

5.6 OSC Bridge and TouchDesigner Input

The frontend will connect to the `touch_designer/osc_bridge.py` over WebSocket at `ws://localhost:7011`. Once the analysis is complete, it sends a JSON payload containing file path, duration, chunk duration, arrays for valence, arousal, tempo, energy, brightness, dominant mood index, six mood-probability arrays, and optional stem paths. The bridge converts this JSON into OSC messages sent to TouchDesigner on UDP port 7000.

TouchDesigner receives the OSC stream with two operator types. An OSC In DAT named `oscin_ctrl` receives control messages such as file paths, stems, transport, and seek. An OSC In CHOP named `oscin_timeline` receives the timeline arrays as multi-sample channels. This separation is useful because control messages behave like events, while timeline values behave like channels.

Table 2: Main TouchDesigner operators used by the final visualization setup.

Operator type	Name	Purpose
OSC In DAT	<code>oscin_ctrl</code>	Receives file paths, stem paths, metadata, transport, and seek events.
OSC In CHOP	<code>oscin_timeline</code>	Receives multi-sample valence, arousal, feature, dominant mood, and mood-probability arrays.
Audio File In CHOP	<code>audiofilein1</code>	Plays the full mixed song.
Audio File In CHOPs	<code>audiofilein_vocals</code> , etc.	Load and play each Demucs stem in sync with the main audio.
Timer CHOP	<code>playhead_timer</code>	Provides the current song time for timeline lookup.
Script CHOP	<code>current_mood</code>	Samples the timeline at the current playhead and outputs one-sample control channels.
Analyze/Math/Lag CHOPs	stem control branches	Convert each stem into smoothed visual control values.

The `current_mood` Script CHOP is the key link between the precomputed model timeline and the live TouchDesigner network. It reads the current playhead time, divides it by the received `chunk_duration`, clamps the index to the valid timeline range, and outputs channels such as `valence`, `arousal`, `energy`, `brightness`, `happy`, `calm`, `romantic`, and `confidence`. TouchDesigner parameters can then reference these channels directly.

5.7 TouchDesigner Visual Mapping

The TouchDesigner project file is `touch_designer/Ghost_Flower.95.toe`. The visual design is a flower-like central form surrounded by moving particle fields. The design uses two kinds of control signals. The MuQ-BiGRU timeline controls emotional state: valence and arousal affect color, brightness, and movement intensity, while mood probabilities affect palette blending. The Demucs stems control instrument-specific motion.

Table 3: Stem-to-visual mapping used for the flower visualization.

Stem	Visual role
Vocals	Flower center glow, petal opening, and emotional pulse.
Drums	Particle bursts, sharp bloom hits, and quick light flashes.
Bass	Whole-flower scale, core expansion, and low-frequency breathing.
Guitar	Particle swirl, noise turbulence, and orbit speed.
Piano	Petal shimmer, sparkle amount, and gentle twist.
Other	Background atmosphere, slow field motion, and residual texture.

The color system uses mood probabilities instead of only one dominant label. This helps avoid sudden color changes when two moods are close in probability. The final color workflow uses the `current_mood` channels, optional Lag CHOP smoothing, Ramp TOP

color controls, Lookup TOP, HSV Adjust, Level, and Luma Blur. In practice, valence and arousal define the broad emotional region, while mood probability weights decide how strongly each mood palette contributes.

5.8 TouchDesigner Render Streaming

The browser does not render the TouchDesigner scene directly. Instead, TouchDesigner runs `touch_designer/td_mjpeg_server.py`. The script captures the final output TOP, converts each frame to JPEG, and serves it as a multipart MJPEG stream. The frontend displays `http://localhost:9000/video` in the main panel when available.

This design keeps the visual runtime and the browser UI separate. TouchDesigner can use its native SOP, CHOP, TOP, lighting, camera, and render pipeline, while the browser remains responsible for interaction, playback controls, saved analyses, and high-level status display.



Figure 6: Final MoodWave frontend and TouchDesigner demonstration

6 Experiments and Results

6.1 Experimental Setup

The final comparison uses the strict DEAM split. This means the test set contains songs that were not seen during training. MuQ embeddings are aligned with the 0.5-second dynamic labels. The MuQ-MLP baseline receives one window embedding at a time, while MuQ-BiGRU receives the full ordered sequence of embeddings from each song. During evaluation, only real windows are scored; padded positions are ignored.

Table 4 summarizes the main hyperparameters used for the final MuQ-based experiments. Both models use the same optimizer family and learning-rate scale, but they

differ in how they consume the MuQ embeddings. MuQ-MLP treats each window independently, while MuQ-BiGRU uses a recurrent layer to model the complete sequence of windows from each song.

Table 4: Main hyperparameters used for the final MuQ-based training runs.

Hyperparameter	MuQ-MLP	MuQ-BiGRU
Input format	Independent 0.5s window embedding	Song sequence of 0.5s embeddings
Optimizer	AdamW	AdamW
Learning rate	1×10^{-3}	1×10^{-3}
Weight decay	1×10^{-4}	1×10^{-4}
Batch size	64	32
Hidden structure	MLP layers (512, 256)	BiGRU hidden size 128
Number of recurrent layers	–	2
Dropout	0.3	0.5
Maximum epochs	80	80

For the LSO experiments, the search was restricted to practical ranges so that candidate models could be evaluated within a reasonable time. Table 5 lists the MuQ-specific search spaces used by the implementation.

Table 5: LSO search space for MuQ-MLP and MuQ-BiGRU.

Parameter	MuQ-MLP search space	MuQ-BiGRU search space
Learning rate	$\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$	$\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$
Weight decay	$\{10^{-5}, 10^{-4}, 10^{-3}\}$	$\{10^{-5}, 10^{-4}, 10^{-3}\}$
Batch size	$\{32, 64, 128\}$	$\{8, 16, 32\}$
Model size	Hidden layers: (256, 128), (512, 256), (512, 256, 128)	Hidden size: $\{64, 128, 256\}$
Number of layers	–	$\{1, 2\}$
Dropout	Continuous range $[0.1, 0.5]$	Continuous range $[0.1, 0.6]$

6.2 Quantitative Results

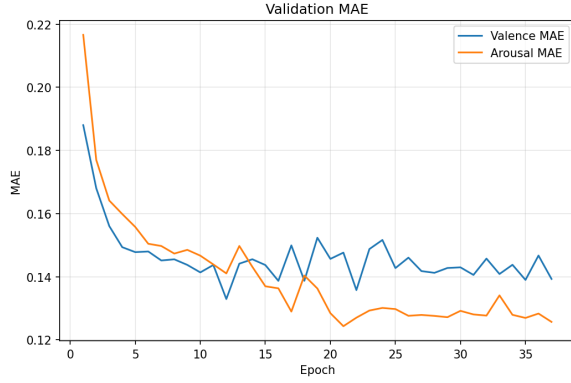
Table 6 summarizes the final strict-split test results produced by the backend.

Table 6: Strict-split DEAM test performance for the evaluated models. Higher R^2 and quadrant accuracy are better; lower MAE is better.

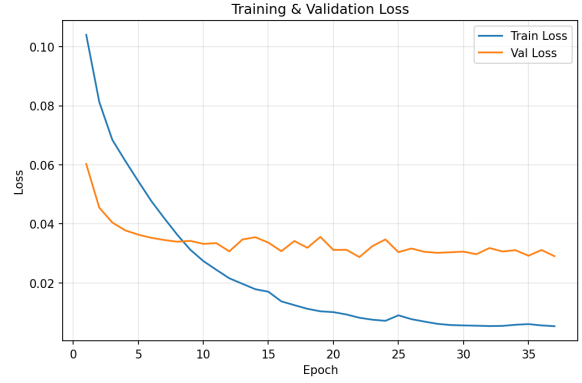
Model	V-MAE	V- R^2	A-MAE	A- R^2	Q-Acc
CNN-BiGRU	0.151	0.399	0.142	0.559	0.616
OpenL3-MLP	0.161	0.322	0.139	0.606	0.592
CLAP-MLP	0.164	0.287	0.149	0.563	0.586
MuQ-MLP	0.129	0.552	0.126	0.667	0.642
MuQ-MLP-LSO	0.129	0.544	0.125	0.670	0.646
MuQ-BiGRU-LSO	0.138	0.512	0.143	0.572	0.644
MuQ-BiGRU	0.124	0.603	0.119	0.680	0.674

Key findings from Table 6:

1. **MuQ-BiGRU is the best overall model.** It achieves the best score for every metric shown in the table where lowest valence MAE, lowest arousal MAE, highest valence R^2 , highest arousal R^2 , and highest quadrant accuracy. This shows that the final sequence-based design is stronger than treating windows independently.
2. **Pre-trained MuQ embeddings clearly improve over earlier encoders.** CNN-BiGRU, OpenL3-MLP, and CLAP-MLP are weaker than the MuQ-based models under the strict split. This suggests that MuQ provides a more useful music representation for DEAM dynamic valence-arousal prediction.
3. **Temporal modeling helps when the input is a real sequence.** MuQ-MLP is already strong because each MuQ embedding contains rich musical information. In addition, MuQ-BiGRU improves over MuQ-MLP where valence R^2 increases from 0.552 to 0.603, arousal R^2 from 0.667 to 0.680, and quadrant accuracy from 0.642 to 0.674. This shows that surrounding windows provide useful context for predicting the emotion of the current window.
4. **Arousal is easier than valence.** Most models have higher arousal R^2 than valence R^2 . This is expected because arousal is closely related to acoustic energy, rhythm, and intensity, while valence depends on more subtle musical cues.
5. **LSO did not guarantee better test performance.** The LSO versions of MuQ-MLP and MuQ-BiGRU did not outperform the final non-LSO MuQ-BiGRU. This shows that the search result can overfit to the validation split or select settings that work during short candidate evaluations but do not generalize as well after full training.

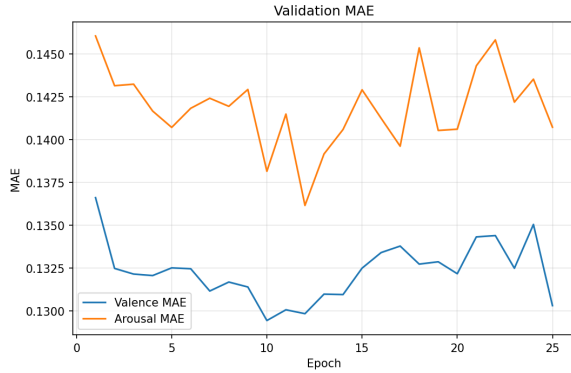


(a) MuQ-BiGRU MAE curve



(b) MuQ-BiGRU loss curve

Figure 7: MAE and loss curves for MuQ-BiGRU.



(a) MuQ-MLP MAE curve



(b) MuQ-MLP loss curve

Figure 8: MAE and loss curves for MuQ-MLP.

Findings from the training curves:

1. **MuQ-BiGRU learns steadily and generalizes better.** In Figure 7, the training loss decreases smoothly, while the validation loss drops early and then stabilizes instead of increasing sharply. This suggests that the model is learning useful temporal patterns without severe overfitting. The MAE curve also shows that arousal error continues to improve and becomes lower than valence error, which matches the final test metrics.
2. **MuQ-MLP converges quickly but has limited improvement after early epochs.** In Figure 8, the training loss continues to decrease, but the validation loss stays almost flat after the first few epochs. This means the MLP fits the training data better over time, but the extra fitting does not translate into better validation performance. The model is useful as a baseline, but its independent-window design limits how much it can improve.

6.3 Discussion

The results lead to three main conclusions. First, pre-trained music encoders are valuable for this task. DEAM is useful, but it is not large enough to train a strong music

representation from scratch. MuQ gives the model a better starting point because it was pre-trained on music audio before being used for valence-arousal prediction.

Second, dynamic prediction is the right formulation for MoodWave. A track-level average can describe the general mood of a song, but it cannot describe when the song becomes brighter, calmer, sadder, or more energetic. For a visualization system, those internal changes are exactly what matter.

Third, recurrence helps only when the input actually contains temporal structure. A BiGRU is not automatically better than an MLP. It becomes useful when the input is a real sequence of windows from the same song. In the final MuQ-BiGRU design, the recurrent layer can compare each window with its surrounding context, which explains why it improves over the independent-window baseline.

The model still performs better on arousal than on valence. This is expected in music emotion recognition. Arousal is often related to more direct acoustic cues such as energy, tempo, onset density, and spectral brightness. Valence is more subtle. It can depend on harmony, mode, lyrical meaning, and cultural expectations, which are harder to infer from audio features alone.

7 Limitations and Future Work

7.1 Deployment-Time MuQ Memory

The most important engineering limitation is MuQ inference memory. During training, DEAM excerpts are short enough to process with cache and fallback handling. Real user songs, however, can be several minutes long. Sending a full song through MuQ at once can cause CUDA out-of-memory errors.

The practical solution is chunked MuQ extraction. The backend should process 10–20 second overlapping audio chunks, pool them into 0.5-second embeddings, merge the embeddings in chronological order, and then run MuQ-BiGRU over the final sequence. This keeps GPU memory usage predictable while preserving the dynamic prediction timeline.

7.2 Train-Inference Consistency

The training and inference pipelines should use the same feature extraction strategy. If inference changes from full-song MuQ extraction to chunked MuQ extraction, the model may receive slightly different embeddings from the ones it saw during training. This mismatch can reduce performance. Therefore, the training cache should eventually be regenerated with the same chunked extraction method used in deployment, and the model should be retrained or at least re-evaluated under that setting.

7.3 Hyperparameter Search Reliability

LSO provides an automated way to explore hyperparameters, but the experiments show that a larger search does not guarantee a better final test score. A search process can overfit to one validation split, especially when many candidate models are compared.

Future work should make the search more robust by always including the non-LSO baseline as a protected candidate, using repeated validation splits when possible, and selecting models based on a metric aligned with the final goal, such as average validation R^2 or a composite score that also includes macro F1.

7.4 Cross-Dataset Evaluation

The final system is evaluated primarily on DEAM. This is appropriate for development, but a stronger research evaluation would also test the model on another dynamic MER dataset such as PMEmo. Cross-dataset testing would show how well the model handles different annotation protocols, genres, and audio quality. This matters because a real visualization system will receive music outside the DEAM distribution.

7.5 Frontend and TouchDesigner Synchronization

The final demo runs across several local processes: FastAPI backend, OSC bridge, browser frontend, and TouchDesigner. This makes the system flexible, but it also introduces synchronization issues. The browser audio player is currently treated as the playback reference, while TouchDesigner follows through OSC seek and transport messages. A small fixed offset is used to compensate for visual and audio latency. Future work could make this more robust by adding round-trip timing measurement, better clock synchronization, or a shared playback clock exposed by TouchDesigner.

7.6 Visualization Evaluation

The backend can be evaluated with numerical MER metrics, but the TouchDesigner output is more subjective. The current flower visualization is designed manually by mapping moods, stems, and audio features to visual parameters. Future work should include user studies or structured visual evaluation, for example asking whether viewers can identify mood changes from the visualization alone, or comparing mixed-audio reactivity against stem-based reactivity. Evaluation from a rock vocalist also mention that the visualization is too cluttered, so that must be improved as well.

8 Conclusion

MoodWave evolved from a basic audio-emotion prototype into a complete dynamic MER and visualization system. The most important technical change was moving from track-level thinking to window-level sequence prediction. Instead of asking, “What is the mood of this song?”, the final system asks, “How does the mood of this song change over time?” This question better matches the needs of an audio-reactive visualization.

The final MuQ-BiGRU model achieves the strongest strict-split window-level results among the implemented MuQ variants, with valence $R^2 = 0.603$, arousal $R^2 = 0.680$, and quadrant accuracy 67.4%. Beyond the model itself, the project now includes a unified training script, organized artifact storage, reusable evaluation code, browser frontend, saved-analysis library, Demucs stem separation, an OSC bridge, and a TouchDesigner flower visualization. The next step is to make MuQ inference production-ready for full-length songs through chunked extraction, improve synchronization between browser audio and TouchDesigner, and improve the visual experience for users.

References

- [1] J. A. Russell, “A circumplex model of affect,” *Journal of Personality and Social Psychology*, vol. 39, no. 6, pp. 1161–1178, 1980.
- [2] M. Soleymani, A. Aljanaki, and Y.-H. Yang, “DEAM: MediaEval database for emotional analysis in music,” University of Geneva and Academia Sinica, 2018.
- [3] A. Cramer, H.-H. Wu, J. Salamon, and J. P. Bello, “Look, listen, and learn more: Design choices for deep audio embeddings,” in *Proc. IEEE ICASSP*, pp. 3852–3856, 2019.
- [4] B. Elizalde et al., “CLAP: Learning audio concepts from natural language supervision,” in *Proc. IEEE ICASSP*, pp. 1–5, 2023.
- [5] Q. Zhu et al., “Self-supervised music representation learning with mel residual vector quantization,” *arXiv preprint arXiv:2501.01108*, 2025.
- [6] A. Defossez, “Hybrid spectrogram and waveform source separation,” *arXiv preprint arXiv:2111.03600*, 2021.